

PLANIFICADORES

SISTEMAS OPERATIVOS

2ª evaluación

Módulo DAW
Desarrollo de Aplicaciones Web
2017/2018

GESTIÓN DE PROCESOS

1. Introducción

En lo que respecta a la gestión de procesos, el SO debe ser capaz de soportar:

- La creación, suspensión, reanudación, eliminación y planificación de procesos
- La comunicación y sincronización entre procesos.

2. Procesos

Un **proceso** es un **programa en ejecución** que necesita estar **cargado en memoria** y disponer de **recursos** (CPU, memoria, archivos, dispositivos de E/S) para cumplir su objetivo. Se trata de una entidad activa. Mientras que los **programas** son un conjunto de **archivos** que están **almacenados** en algún **dispositivo de almacenamiento** (disco duro, pendrive...) y cuyo código fuente está escrito en algún lenguaje de programación. Cuando este conjunto de archivos se ejecutan, entonces pasan a ser un proceso.

2.1 Estados

El estado de un proceso se define por su actividad actual, cambiando a medida que se ejecuta. La ejecución de un proceso alterna una serie de ráfagas de CPU y E/S.



Transiciones más importantes:

- Nuevo-Preparado: el SO está preparado para admitir un proceso más.
- Preparado-Ejecución: El planificador escoge un proceso para la ejecución.
- Ejecución-Preparado: El proceso en ejecución es interrumpido y expulsado del procesador porque ya ha consumido su tiempo asignado o porque otro proceso de mayor prioridad está esperando.
- Ejecución-Bloqueado: El proceso abandona voluntariamente la CPU y espera a un evento externo.
- Bloqueado-Preparado: Finaliza el evento que estaba esperando el proceso y pasa al estado preparado.
- Ejecución-Terminado: El proceso termina su ejecución (terminación normal)
- Preparado/Bloqueado-Terminado: El proceso es eliminado (terminación anormal).

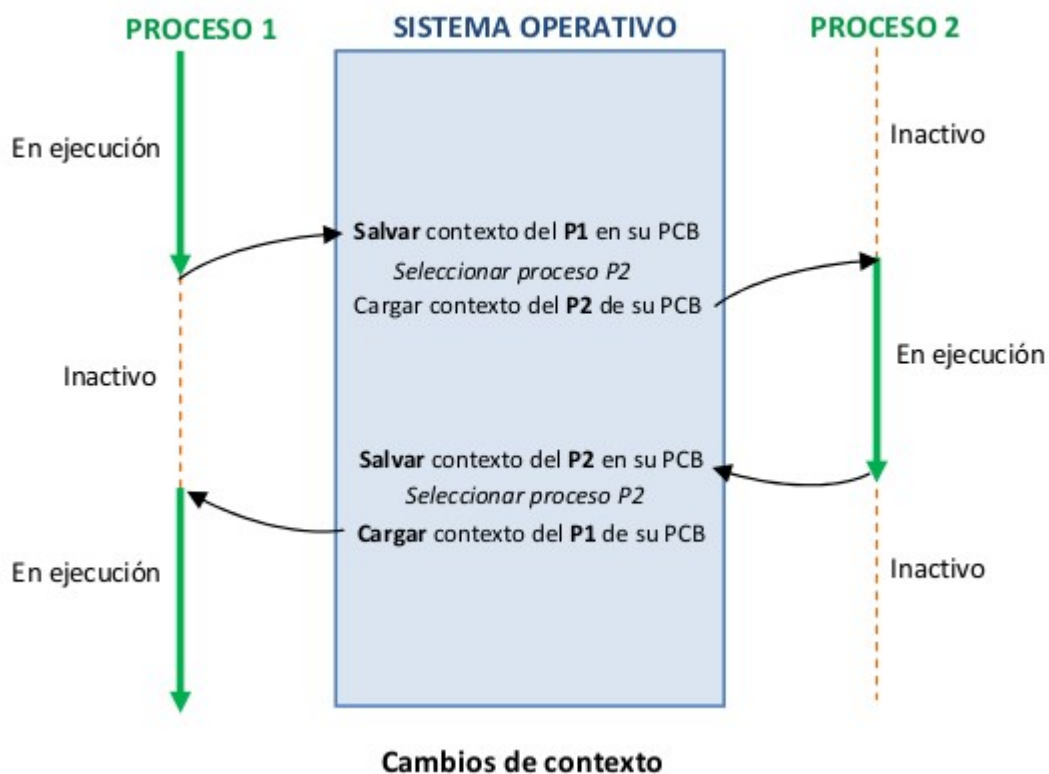
2.2 Implementación

Cuando un proceso se ejecuta, el SO le asigna un espacio de direcciones de memoria (que contiene las instrucciones, los datos y la pila (estructura para almacenar y recuperar datos) del proceso) y lo añade a una **tabla de procesos**.

El S.O guarda en la tabla de procesos **por cada proceso** una estructura de datos llamada **Bloque de Control de Proceso** (PCB) que almacena la siguiente información:

- **Identificación** de proceso (del proceso en sí (PID), del proceso padre(PPID) y de usuario)
- **Información de estado** del proceso: preparado, en ejecución, bloqueado...
- **Prioridad** del proceso
- **Dirección de memoria** donde se ha cargado el proceso
- **Otros** (recursos utilizados, valores de los registros del procesador, propietario, permisos...)

Para dar sensación de ejecución simultánea o **multiprogramación**, el tiempo de CPU debe repartirse entre los procesos. Esto implica **cambios de contexto** que consisten en quitarle la CPU al proceso “en ejecución” y asignársela a otro en estado “preparado”. Esta operación la realiza un componente del S.O. llamado dispatcher o **planificador a corto plazo** y en ella se guarda el contexto del proceso en ejecución en su PCB y se restaura el contexto del nuevo proceso a ejecutar mediante su PCB.



Los cambios de contexto pueden suponer una sobrecarga si se utilizan con mucha frecuencia. En general, suelen producirse cuando un proceso finaliza, es expulsado o se suspende.

2.3 Hilos

En los actuales S.O's un proceso puede tener internamente varias actividades concurrentes llamadas hilos de ejecución o threads.

Son como “miniprosesos”, unidades pequeñas en las que se divide un proceso, cada una de las cuales realiza una acción.

(Diferencias entre hilos y procesos)

Los hilos se distinguen de los tradicionales procesos en que los procesos son (generalmente) independientes, llevan bastante información de estados, e interactúan sólo a través de mecanismos de comunicación dados por el sistema. Por otra parte, muchos hilos generalmente comparten otros recursos de forma directa. En muchos de los sistemas operativos que dan facilidades a los hilos, es más rápido cambiar de un hilo a otro dentro del mismo proceso, que cambiar de un proceso a otro. Este fenómeno se debe a que los hilos comparten datos y espacios de direcciones, mientras que los procesos, al ser independientes, no lo hacen. Al cambiar de un proceso a otro el sistema operativo (mediante el dispatcher) genera lo que se conoce como overhead, que es tiempo desperdiciado por el procesador para realizar un cambio de contexto (context switch), en este caso pasar del estado de ejecución (running) al estado de espera (waiting) y colocar el nuevo proceso en ejecución. En los hilos, como pertenecen a un mismo proceso, al realizar un cambio de hilo el tiempo perdido es casi despreciable

2.4 Procesos y servicios

Los procesos que se ejecutan en un ordenador pueden estar en primer o segundo plano. Los de **primer plano** interactúan con los usuarios, mientras que los de **segundo plano** realizan una función específica sin que interactúe el usuario. Además, los procesos pueden pertenecer al usuario o ser propios del SO.

De este modo, el sistema operativo puede crear procesos, algunos de los cuales residen en **segundo plano** y se llaman **servicios en Windows** y **demonios en Linux**. El administrador del sistema puede iniciar, detener, pausar, reanudar... estos servicios. Los procesos que pertenecen al sistema se ejecutan en el modo kernel o modo privilegiado (podrán acceder a cualquier recurso).

Otros procesos son los que **crea el usuario**, donde éste **interactúa** con ellos, es decir, se ejecutan en **primer plano**, como, por ejemplo, el Firefox. Estos procesos se ejecutan en el llamado “modo usuario” (con restricciones de acceso a los recursos hardware).

3. Planificación de procesos

Como hemos comentado, en un S.O. multitarea la **multiprogramación** se consigue mediante la ejecución alternativa de procesos.

La parte del S.O. encargada de seleccionar el siguiente proceso para ejecutar es el **planificador** y el algoritmo que usa es el **algoritmo de planificación**.

Políticas principales para la planificación:

- **Planificación no apropiativa:** (algoritmos no expulsivos)
 - Los procesos se ejecutan hasta que terminan o se bloquean.
 - Sencillo de implementar.
 - Rendimiento negativo en general.
- **Planificación apropiativa:** (algoritmos expulsivos)
 - Los procesos pueden ser expulsados de la CPU.
 - Mayor coste de implementación. Necesitan soporte hardware adicional (relojes)
 - Mejora el servicio y evita monopolización de la CPU.

3.2 Algoritmos de planificación

Eligen el siguiente proceso para entrar en la CPU. Los principales algoritmos de planificación son:

- **FCFS (First Come First Served)** → cola del supermercado
La CPU es asignada a los procesos en el mismo orden que lo solicitan. Es un algoritmo no expulsivo.

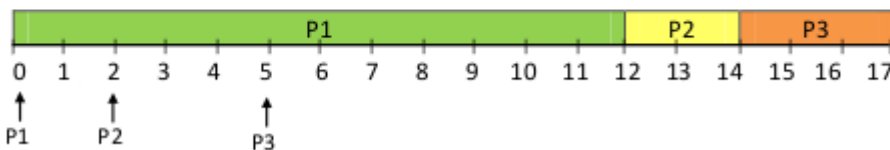
Ventajas:

- Sencillo de implementar (cola FIFO).

Inconvenientes:

- Mal tiempo medio de espera.
- Efecto convoy (procesos con largas ráfagas de CPU retrasan a procesos con ráfagas cortas).
- No válido para procesos interactivos.

Proceso	Instante llegada	Tiempo CPU
P1	0	12
P2	2	2
P3	5	3



$$\text{Tiempo medio de espera} = (0+10+9)/3=6,3$$

FIFO: First In First Out (primero en entrar, primero en salir) es un concepto utilizado en estructura de datos y teoría de colas. Guarda analogía con las personas que esperan en una cola y van siendo atendidas en el orden en que llegaron, es decir, la primera persona que entra es la primera persona que sale.

- **SJF (Shortest Job First)** → primero el que menos tiempo total de CPU requiere
Se escoge el proceso de la cola de preparados con una próxima racha de CPU más corta y se ejecuta hasta que se termine o se suspenda. Si hay varios procesos con rachas de CPU iguales, se puede aplicar FIFO. **Algoritmo no expulsivo.**

Ventajas:

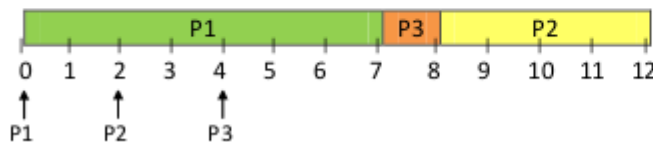
- Optimiza el tiempo de espera
- Favorece los procesos orientados a E/S

Desventajas:

- Es costoso averiguar cuánto dura la siguiente racha de CPU
- Inanición de los procesos con rachas de CPU largas.

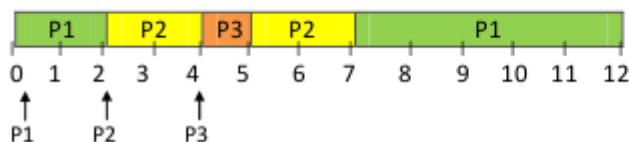
Proceso	Instante llegada	Tiempo CPU
P1	0	7
P2	2	4
P3	4	1

$$\text{Tiempo medio de espera} = (0+6+3)/3 = 3$$



- **SRTF (Shortest Remaining Time First)** → primero al que menos tiempo de CPU le queda para acabar.
Versión apropiativa de SJF (como el SJF, solo que se puede echar a los procesos).
Algoritmo expulsivo.

Proceso	Instante llegada	Tiempo CPU
P1	0	7
P2	2	4
P3	4	1



$$\text{Tiempo medio de espera} = (5+1+0)/3 = 2$$

- **Planificación por prioridades** → primero el que tiene más prioridad

Cada proceso tiene asignada una prioridad. El planificador selecciona el proceso con mayor prioridad. En caso de igual prioridad se selecciona con FCFS.

Las prioridades pueden ser dinámicas (cambian con el tiempo) o estáticas (se mantienen).

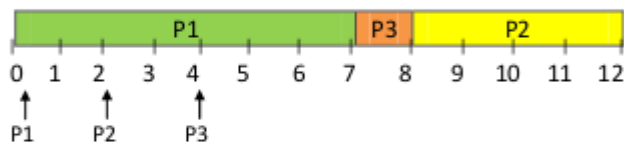
Inconvenientes:

- Riesgo de inanición de procesos con prioridad baja. Una solución sería aumentar la prioridad con el incremento del tiempo de espera.

(en el ejemplo tomaremos como mayor prioridad la del número más alto, has de saber que no siempre es así, podría ser mayor prioridad el número más bajo)

Proceso	Instante llegada	Tiempo CPU	Prioridad
P1	0	7	5
P2	2	4	10
P3	4	1	15

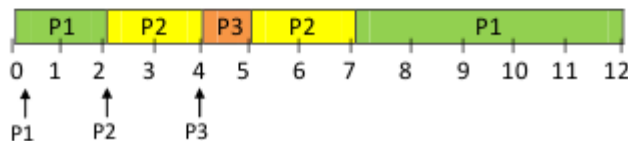
Prioridades sin expulsión:



Tiempo medio de espera:

$$(0+6+3)/3=3$$

Prioridades con expulsión:



Tiempo medio de espera =

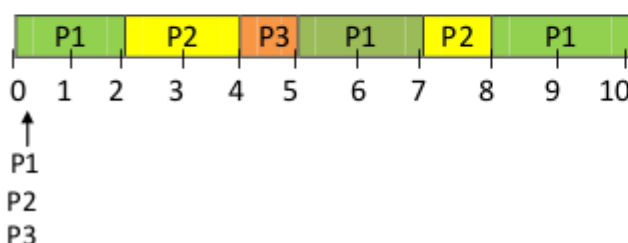
$$(5+1+0)/3 = 2$$

- **Planificación circular (Round Robin)** → todos el mismo tiempo por turnos

- A cada proceso se le permite una cantidad de tiempo para usar la CPU llamada "**quantum**". Si el proceso necesita un tiempo de CPU mayor que el quantum, entonces será expulsado de la CPU.
- La cola de preparados se gestiona con una política FIFO.
- Si el valor de quantum es grande el algoritmo degenera en FCFS. Si es pequeño se generará una sobrecarga debido a cambios de contexto.
- Es equitativo

Ej. (Quantum = 2):

Proceso	Instante llegada	Tiempo CPU
P1	0	6
P2	0	3
P3	0	1



Tiempo medio de espera=

$$(4+5+4) / 3 = 4,3$$